

NAME : ANSAR ULLAH SIDDIQUI

EB22210006021

KUDCS-UBIT , SEC-B , EVENING

COURSE : INTERNET APP DEVELOPMENT **BSCS-633**

Software Documentation

1. Introduction

This software, named **JobApp** (also referred to as **JobSeekingApp**), is a web-based Job Portal application. Its primary purpose is to facilitate the management and viewing of job postings.

The application is designed for two main types of users:

- **Job Posters (e.g., Employers, Recruiters):** These users can add new job listings to the platform. Access to this functionality is secured, requiring authentication.
- **Job Seekers:** These users can browse and view all available job postings on the platform.

2. Getting Started

To get the Job Portal application up and running, follow these steps:

2.1 Prerequisites

Ensure you have the following software installed on your system:

- **Java Development Kit (JDK) 21 or higher:** The application is built with Java 21. You will need a compatible Java Runtime Environment (JRE) to run it.
- **Apache Maven:** Used for building and managing the project.
- **PostgreSQL Database:** The application uses PostgreSQL as its database.

2.2 Database Setup

1. **Install PostgreSQL:** If you don't have PostgreSQL installed, download and install it from the official PostgreSQL website.
2. **Create Database:** Create a new database named LearnSpring in your PostgreSQL instance. You can do this using a PostgreSQL client (like pgAdmin) or via the psql command line:
`CREATE DATABASE "LearnSpring";`

3. **Configure Database Credentials:** The application expects the database username to be postgres and the password to be 0000. If your PostgreSQL setup uses different credentials, you will need to update the src/main/resources/application.properties file:
spring.datasource.username=your_postgres_username
spring.datasource.password=your_postgres_password

2.3 Building and Running the Application

1. **Clone the Repository:** Obtain the source code for the JobApp project.
2. **Navigate to Project Directory:** Open your terminal or command prompt and navigate to the root directory of the JobApp project (where the pom.xml file is located).
3. **Build the Project:** Use Maven to build the project. This will download dependencies and compile the code:
mvn clean install
4. **Run the Application:** Once the build is successful, you can run the Spring Boot application:

`"mvn spring-boot:run"`

Alternatively, you can run the generated JAR file from the target directory:

`"java -jar target/JobApp-0.0.1-SNAPSHOT.jar"`

2.4 Accessing the Application

Once the application starts successfully (you'll see Spring Boot logs indicating it's running on port 8080), open your web browser and navigate to:

http://localhost:8080/ or http://localhost:8080/home

2.5 Default Login Credentials

The application is secured with basic authentication. Use the following default credentials to log in:

- **Username:** xyz
- **Password:** 1234

3. Features

The Job Portal application provides the following core functionalities:

- **User Authentication:** The application is secured using Spring Security, requiring users to authenticate (log in) before accessing most of its features.
- **Job Posting:** Authenticated users can add new job listings, providing details such as post ID, profile, description, required experience, and a tech stack.
- **View All Job Listings:** Users can browse and view a comprehensive list of all available job postings. Each job listing displays its profile, description, required experience, and associated tech stack.
- **Database Persistence:** All job postings are stored persistently in a PostgreSQL database, ensuring data is saved even after the application restarts.
- **Web-Based User Interface:** The application provides a user-friendly web interface built with JavaServer Pages (JSPs) and styled using Bootstrap 5, offering a responsive and intuitive experience.

4. Technical Details

Your software, named **JobApp** (or **JobSeekingApp** as per application.properties), is built using **Java Spring Boot**. It's a web application designed to manage job postings.

4.1 Project Structure and Dependencies (pom.xml)

The project is a Maven-based Spring Boot application with the following key dependencies:

- **Spring Boot Starter Parent (version 3.2.0):** Provides sensible defaults for Spring Boot projects.
- **Java Version:** 21
- **spring-boot-starter-web:** Enables building web applications, including RESTful services.
- **spring-boot-starter-data-jpa:** Provides Spring Data JPA for easy integration with relational databases using Hibernate.
- **postgresql:** JDBC driver for PostgreSQL database, used as the runtime database.

- **tomcat-jasper:** Enables JSP support for rendering dynamic web pages.
- **spring-boot-devtools:** Provides development-time features like automatic restarts.
- **lombok:** A library that helps reduce boilerplate code (e.g., getters, setters, constructors).
- **jakarta.servlet.jsp.jstl-api & jakarta.servlet.jsp.jstl:** Dependencies for JavaServer Pages Standard Tag Library (JSTL) support.
- **spring-boot-starter-security:** Integrates Spring Security for authentication and authorization.
- **spring-security-taglibs:** Provides JSP tag libraries for Spring Security.
- **spring-boot-starter-test:** For writing unit and integration tests.

The pom.xml also includes the spring-boot-maven-plugin for building executable JARs.

4.2 Application Configuration (application.properties)

The application.properties file configures various aspects of the application:

- **spring.application.name=JobSeekingApp:** Defines the application's name.
- **spring.mvc.view.prefix=/views/:** Specifies the prefix for JSP views.
- **spring.mvc.view.suffix=.jsp:** Specifies the suffix for JSP views. This means JSP files are expected in the /src/main/webapp/views/ directory.
- **Database Configuration (PostgreSQL):**
 - `spring.datasource.url=jdbc:postgresql://localhost:5432/LearnSpring`
 - `spring.datasource.username=postgres`
 - `spring.datasource.password=0000`
 - `spring.datasource.driver-class-name=org.postgresql.Driver`

The application connects to a PostgreSQL database named LearnSpring running locally on port 5432.
- **spring.jpa.hibernate.ddl-auto=update:** Configures Hibernate to update the database schema based on JPA entities.
- **server.port=8080:** The application will run on port 8080.
- **Spring Security User:**
 - `spring.security.user.password=1234`
 - `spring.security.user.name=xyz`

These define a default in-memory user for basic authentication.
- **logging.level.org.springframework.web=INFO:** Sets the logging level for Spring web components to INFO.

4.3 Application Components

4.3.1 JobController

This is the main Spring MVC controller handling web requests:

- **@Autowired private JobService js;**: Injects the JobService to interact with business logic.
- **@GetMapping("/{}/","home")**: Maps requests to the root URL or /home to display the home.jsp page.
- **@GetMapping("addjob")**: Displays the addjob.jsp form for posting new jobs.
- **@PostMapping("handleForm")**: Handles the submission of the job posting form. It receives a JobPost object and uses JobService to save it, then redirects to a success.jsp page.
- **@GetMapping("viewalljobs")**: Fetches all job posts from the JobService and passes them to the viewalljobs.jsp for display.

4.3.2 SecurityConfig

This class configures Spring Security for the application:

- **@Configuration & @EnableWebSecurity**: Annotations to enable and configure Spring Security.
- **defaultSecurityFilterChain(HttpSecurity http)**: Defines the security rules:
 - `http.authorizeHttpRequests((requests) -> requests.anyRequest().authenticated());`: All requests require authentication.
 - `http.formLogin(withDefaults());`: Enables form-based login with default settings.
 - `http.httpBasic(withDefaults());`: Enables HTTP Basic authentication.

4.3.3 JobService

This class handles the business logic related to job posts:

- **@Service**: Marks this class as a service component in Spring.
- **@Autowired private JobRepo repo;**: Injects the JobRepo (JPA Repository) to interact with the database.
- **addjob(JobPost jp)**: Saves a JobPost object to the database using the JobRepo.
- **viewJobs()**: Retrieves all JobPost objects from the database.

4.3.4 JobRepo (JPA Repository)

This is a Spring Data JPA repository interface:

- **@Repository**: Marks this interface as a repository component.
- **extends JpaRepository<JobPost , Integer>**: Extends JpaRepository, providing standard CRUD (Create, Read, Update, Delete) operations for JobPost entities,

with Integer as the type of the primary key.

4.4 User Interface (JSPs) and Styling

The application uses JavaServer Pages (JSPs) for its front-end, styled with **Bootstrap 5** and custom CSS files (style.css, style1.css).

- **home.jsp**: The landing page, providing links to "View All Jobs" and "Add Job". It includes a navigation bar.
- **addjob.jsp**: A form for users to post new job listings. It includes fields for Post ID, Post Profile, Post Description, Required Experience, and a multi-select for Tech Stack. It also includes Spring Security's CSRF token (<sec:csrfInput />).
- **success.jsp**: (Implicitly mentioned by handleForm returning "success") - This page would typically confirm the successful submission of a job post.
- **viewalljobs.jsp**: Displays a list of all available job posts in a card format, showing the profile, description, required experience, and tech stack for each.
- **style.css and style1.css**: Custom CSS files are used for additional styling beyond Bootstrap defaults.

5. Troubleshooting / FAQs

This section outlines common issues you might encounter while setting up or running the Job Portal application, along with their potential solutions.

5.1 Database Connection Issues

- **Issue**: The application fails to connect to the PostgreSQL database, often resulting in org.postgresql.util.PSQLException or Cannot create PoolableConnectionFactory errors in the logs.
 - **Reason**: Incorrect database URL, username, password, or the PostgreSQL server is not running.
 - **Solution**:
 1. **Verify PostgreSQL Status**: Ensure your PostgreSQL server is running.
 2. **Check application.properties**: Double-check the spring.datasource.url, spring.datasource.username, and spring.datasource.password in src/main/resources/application.properties for typos or incorrect values.
 3. **Database Name**: Confirm that the database LearnSpring actually exists in your PostgreSQL instance.
 4. **Firewall**: Ensure no firewall is blocking the connection to port 5432 (or your configured PostgreSQL port).

5.2 Port Conflict

- **Issue**: The application fails to start with an error like Port 8080 was already in

use.

- **Reason:** Another application or service is already using port 8080.
- **Solution:**
 1. **Identify Occupying Process:** On Linux/macOS, use `lsof -i :8080`. On Windows, use `netstat -ano | findstr :8080` and then `taskkill /PID <PID> /F`.
 2. **Change Application Port:** Modify the `server.port` property in `src/main/resources/application.properties` to a different available port (e.g., `server.port=8081`).
 3. **Stop Conflicting Process:** If possible, stop the application that is currently using port 8080.

5.3 Authentication Issues

- **Issue:** Unable to log in with the default credentials, or unexpected redirection/access denied errors.
 - **Reason:** Incorrect username/password, or Spring Security configuration issues.
 - **Solution:**
 1. **Verify Credentials:** Ensure you are using the default username `xyz` and password `1234` as configured in `application.properties`.
 2. **Check SecurityConfig:** Review the `SecurityConfig` class to ensure `http.authorizeHttpRequests((requests) -> requests.anyRequest().authenticated());` and `http.formLogin(withDefaults());` are correctly configured.
 3. **Browser Cache:** Clear your browser's cache and cookies, as old session data can sometimes interfere.

5.4 JSP or Static Resource Not Found (404 Errors)

- **Issue:** Pages (JSPs) or static resources (CSS files like `style.css`, `style1.css`) are not loading, resulting in 404 errors.
 - **Reason:** Incorrect paths, files not deployed correctly, or Spring MVC view resolver issues.
 - **Solution:**
 1. **Verify Paths:** Ensure your JSP files are located in `src/main/webapp/views/`. For example, `home.jsp` should be at `src/main/webapp/views/home.jsp`.
 2. **Check application.properties:** Confirm `spring.mvc.view.prefix=/views/` and `spring.mvc.view.suffix=.jsp` are correctly set.
 3. **Static Resources:** Ensure `style.css` and `style1.css` are placed in `src/main/webapp/` or a dedicated static resources folder (e.g.,

src/main/resources/static). Spring Boot typically serves static content from /static, /public, /resources, or /META-INF/resources. If they are in src/main/webapp/, ensure the relative paths in JSPs are correct (e.g., <link rel="stylesheet" type="text/css" href="style.css">).

4. **Maven Build:** Perform a mvn clean install to ensure all resources are properly packaged in the JAR.

5.5 Maven Build Failures

- **Issue:** The mvn clean install command fails with compilation errors or dependency resolution issues.
 - **Reason:** Missing dependencies, network issues preventing dependency download, or syntax errors in Java code.
 - **Solution:**
 1. **Check Internet Connection:** Ensure you have an active internet connection to download Maven dependencies.
 2. **Review pom.xml:** Verify all dependencies in pom.xml are correctly defined and have valid versions.
 3. **Clean Build:** Always start with mvn clean install to ensure a fresh build.
 4. **IDE Build:** If using an IDE (like IntelliJ IDEA or Eclipse), try building the project through the IDE's Maven integration to get more detailed error messages.
 5. **Java Version:** Confirm your installed JDK version matches the <java.version> specified in pom.xml (Java 21 in this case).

5.6 Data Not Persisting or Table Not Created

- **Issue:** Data saved through the "Add Job" form does not appear after restarting the application, or database tables are not automatically created.
 - **Reason:** spring.jpa.hibernate.ddl-auto setting, or issues with the JobPost entity mapping.
 - **Solution:**
 1. **ddl-auto Setting:** Ensure spring.jpa.hibernate.ddl-auto=update is correctly set in application.properties. For initial setup, you might temporarily set it to create or create-drop to force table creation (be aware this will drop existing data on each restart with create-drop).
 2. **JobPost Entity:** Verify that your JobPost model class is correctly annotated as a JPA @Entity and has a defined @Id field.
 3. **Logs:** Check the application logs for any Hibernate-related errors or warnings during startup, which might indicate issues with entity mapping or database interaction.